Homework — 1.3.1 Compression, Encryption and Hashing — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1 (a) [2] Software installer = **lossless**; photograph on a web page = **lossy**; source-code text files = **lossless**; streamed music = **lossy**. *(2 correct rows per mark.)*

1 (b) [1] Source code must be restored *exactly* as it was — a single changed/missing character (bit) could stop the code compiling or change its meaning; lossy compression permanently discards data (1).

2. [2] Error 1: `3R` should be `4R` — there are four Rs at the start (1). Error 2: `5B` should be `4B` — there are four Bs at the end (1). Correct encoding: `4R 3G 2R 4B` (accept `R4 G3 R2 B4`). *(Check: 4 + 3 + 2 + 4 = 13 pixels = length of the original row.)*

3. [2] There are no runs of repeated values — every pixel differs from its neighbour, so every run has length 1 (1). The encoded output (`1R 1G 1R 1G …`) is therefore *larger* than the original, because each pixel now needs a count **and** a value stored (1).

4. [2] (a) **dictionary**; (b) **index** (accept *reference* if written); (c) **smaller**; (d) **decompression**. *(2 correct blanks per mark. Distractors: key, larger, pixel.)*

5. [3] In the order the rows are printed: **4, 1, 6, 3, 5, 2**. Correct sequence: 1 — Bob generates a key pair; 2 — Bob makes his public key openly available; 3 — Alice obtains a copy of Bob's public key; 4 — Alice encrypts the file with Bob's public key; 5 — the encrypted file is sent across the internet; 6 — Bob decrypts it with his private key. *(2 correct rows per mark.)*

6 (a) [1] Hashing is one-way / cannot be reversed — the original password cannot be recovered/calculated from the stored hash (1).

6 (b) [1] The password entered at login is hashed using the same hash function and the result is compared with the stored hash; if they match, the password is accepted (1).

Part B (6 marks)

7. [2] One clear difference, 1 mark per side, e.g.: a compiler translates the *whole* program into object/machine code before execution, producing an executable file (1); an interpreter translates and executes one statement at a time each run, producing no standalone executable (1). (Also accept: compiler reports all errors after compilation vs interpreter halts at the first error; compiled code executes faster.)

8. [2] The **MAR** holds the address of the memory location to be accessed — during fetch, the address of the next instruction copied from the PC (1). The **MDR** holds the data or instruction that has been fetched from (or is to be written to) that memory location (1).

9. [2] The scheduler allocates processor time to processes / decides which process the CPU executes next (1). One aim (1, any one): make best use of CPU time; ensure all processes get a fair share of processor time; maximise throughput; give acceptable response times (accept naming a scheduling algorithm such as round robin as exemplification of how an aim is met).

Total: 14 + 6 = 20 marks